

Smart Attendance Management System

MarkIn / SAMS

A Project Report Submitted
In Partial Fulfilment of the Requirements
for the Degree of

Bachelor of Technology

in

Computer Science & Engineering

by

Student Name 1 (Roll No. 00000000000)

Student Name 2 (Roll No. 00000000000)

Student Name 3 (Roll No. 00000000000)

Under the Guidance of
Guide Name



Department of Computer Science and Engineering
Institute of Engineering & Technology
Dr. APJ Abdul Kalam Technical University, Uttar Pradesh
May, 2026

Certificate

This is to certify that the project report entitled **Smart Attendance Management System** submitted by ***Student Name 1 (Roll No.)***, ***Student Name 2 (Roll No.)*** and ***Student Name 3 (Roll No.)*** in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering is a record of the bonafide work carried out under the guidance of ***Guide Name***, Department of Computer Science and Engineering, Institute of Engineering & Technology, Lucknow.

The project report has reached the standard required for submission. The work described in this report has not been submitted elsewhere for the award of any other degree or diploma to the best of our knowledge.

Date: *DD Month YYYY*

Guide Signature: *Guide Name*

Department of Computer Science and Engineering

Institute of Engineering & Technology, Lucknow

Declaration

We hereby declare that this project report entitled **Smart Attendance Management System** is our own work carried out as part of the Bachelor of Technology final year project. The report explains the design, implementation, testing and evaluation of a smart attendance platform developed for students and teachers. Any external concepts, technologies and documentation referred to in the report have been acknowledged in the References section.

Student 1
Roll No.

Student 2
Roll No.

Student 3
Roll No.

Acknowledgement

We express our sincere gratitude to our project guide ***Guide Name*** for valuable guidance, review and encouragement throughout this work. We also thank the Department of Computer Science and Engineering, Institute of Engineering & Technology, Lucknow, for providing the academic environment and resources needed to complete the project. We are thankful to our teachers, classmates and family members for their support during requirement analysis, development, testing and documentation.

Abstract

The Smart Attendance Management System is a full-stack attendance management platform designed to reduce manual work, improve transparency and support academic decision-making. The system provides role-based dashboards for students and teachers, class creation and joining, face enrollment, AI-assisted classroom photo attendance, QR attendance, manual attendance, attendance analytics, leave proof upload, exam eligibility planning, email notifications and report export. The backend is built with Express and MongoDB, the frontend is built with React, and a separate FastAPI AI service performs face detection and recognition using InsightFace model packs such as Antelopev2 and Buffalo_L.

Unlike a simple attendance register, the system follows a review-first workflow. When a teacher captures classroom photos, the AI service suggests present and absent students. The result is stored in a Today Attendance Data draft, absentees are notified by email, and the teacher can correct the list before final submission. This reduces false marking and gives students a chance to report mistakes. The project also includes email OTP verification, password reset, profile email verification, leave approval/rejection emails, exam attendance warnings and class cancellation notices. This report explains the complete project from beginner level: problem background, requirements, system design, database structure, workflows, implementation, testing, results, limitations and future scope.

Contents

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
1 Introduction	1
1.1 Project Overview	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope of the Project	3
1.6 Users of the System	3
1.7 Why the Project is Useful	4
1.8 Report Organization	4
2 Literature Review and Existing Systems	5
2.1 Background	5
2.2 Manual Attendance Systems	5
2.3 Digital ERP Attendance Systems	6
2.4 Biometric Attendance Systems	6
2.5 QR Attendance Systems	7
2.6 Face Recognition Concepts	7
2.7 Related Technology Stack	7

2.8	Gap in Existing Solutions	8
2.9	Proposed Direction	8
3	Requirement Analysis	9
3.1	Stakeholders	9
3.2	Functional Requirements	10
3.3	Non-Functional Requirements	12
3.4	User Roles and Permissions	13
3.5	Use Case View	14
3.6	Data Requirements	15
3.7	Input and Output Requirements	16
3.8	Constraints and Assumptions	16
3.9	Requirement Traceability	17
4	System Design and Architecture	19
4.1	Architectural Principles	19
4.2	Overall Architecture	19
4.3	Frontend Design	20
4.4	Backend Design	21
4.5	AI Service Design	21
4.6	Database Design	22
4.7	Today Attendance Data Design	23
4.8	Email Notification Architecture	24
4.9	Security Design	24
4.10	API Design Style	24
5	Implementation Details	26
5.1	Project Structure	26
5.2	Technology Stack	27
5.3	Backend Implementation	27
5.3.1	Authentication and Email Verification	28
5.3.2	Classroom and Membership Implementation	28

5.3.3	Attendance Implementation	28
5.3.4	Today Attendance Draft Implementation	29
5.4	Student Dashboard Implementation	29
5.5	Teacher Dashboard Implementation	30
5.6	Leave and Medical Proof Implementation	30
5.7	Exam Schedule and Eligibility Implementation	31
5.8	Email Service Implementation	31
5.9	AI Attendance Implementation	32
5.10	Frontend Navigation Implementation	32
5.11	Error Handling	33
5.12	Configuration Management	33
6	Features and Working	34
6.1	Landing Page	34
6.2	Registration and Login	34
6.3	Student Dashboard	35
6.4	Student Class Detail Page	36
6.5	Attendance Calculator	36
6.6	Teacher Dashboard	37
6.7	Teacher Attendance Workflow	37
6.8	AI Photo Attendance	38
6.9	Today Attendance Data Section	38
6.10	Leave Proof and Medical Report Workflow	39
6.11	Exam Scheduling and Eligibility	39
6.12	Notification Bell and Separate Pages	40
6.13	Class Cancellation and Reschedule Notice	40
6.14	Assignments and Class Discussion	40
6.15	Feature Summary	40
7	Testing and Results	42
7.1	Testing Strategy	42
7.2	Backend API Test Scenarios	43

7.3	Frontend Workflow Test Scenarios	44
7.4	AI Service Testing	45
7.5	Email Testing	46
7.6	Attendance Draft Testing	46
7.7	Security and Privacy Checks	47
7.8	Result Analysis	47
7.9	Known Limitations Found During Testing	48
7.10	Testing Conclusion	48
8	Deployment, Limitations and Future Scope	49
8.1	Local Development Deployment	49
8.2	Environment Configuration	50
8.3	Production Deployment View	50
8.4	Operational Monitoring	50
8.5	Limitations	51
8.6	Future Scope	51
8.7	Ethical and Privacy Considerations	52
8.8	Conclusion	52
A	Supplementary Material	54
A.1	Sanitized Environment Setup	54
A.1.1	Backend Environment Example	54
A.1.2	AI Service Environment Example	55
A.2	Common Development Commands	55
A.3	API Endpoint Summary	55
A.4	Model and Collection Reference	57
A.5	Sample Email Template Content	58
A.6	Report Figure Inventory	59
A.7	Glossary	59

List of Figures

3.1	High-level user role use cases	15
4.1	Overall MarkIn/SAMS architecture and workflow	20
4.2	Service-level communication	20
4.3	Simplified database/entity relationship diagram	22
4.4	Today Attendance Data draft and finalization flow	23
4.5	Email notification flow	24
5.1	Email verified registration implementation flow	28
6.1	Registration and login working	35
6.2	Student dashboard navigation flow	36
6.3	Teacher attendance workflow	38
6.4	Leave proof review workflow	39
7.1	Attendance draft testing focus	47
8.1	Production deployment view	50

List of Tables

1.1	Primary users of the system	3
2.1	Limitations of manual attendance	6
2.2	Technology stack overview	8
3.1	Stakeholders and their expectations	10
3.2	Functional requirement summary	11
3.3	Non-functional requirements	13
3.4	User roles and permissions	14
3.5	Important data required by SAMS	16
3.6	Requirement traceability matrix	17
4.1	Backend layer responsibilities	21
4.2	Database collection summary	23
4.3	Representative API design examples	25
5.1	Technology stack used in implementation	27
5.2	Attendance status and meaning	29
5.3	Student dashboard data shown to users	30
5.4	Email notification events implemented in the backend	31
6.1	Student dashboard feature matrix	35
6.2	Attendance calculator inputs and outputs	37
6.3	Today Attendance Data actions	39
6.4	Complete feature matrix	40
7.1	Testing levels used for the project	43

7.2	Representative backend API test scenarios	43
7.3	Frontend workflow testing	45
7.4	AI service test concerns	45
7.5	Email notification testing scenarios	46
8.1	Local service deployment summary	49
8.2	Important environment categories	50
8.3	Current limitations	51
8.4	Future enhancement roadmap	52
A.1	Common development commands	55
A.2	API endpoint summary	56
A.3	Model reference	57
A.4	Sample email subjects	59
A.5	Important diagrams included in the report	59
A.6	Glossary	60

Chapter 1

Introduction

1.1 Project Overview

Attendance is one of the most common administrative activities in schools, colleges and training institutions. In many classrooms it is still handled by calling names, signing registers or manually entering presence into a spreadsheet. These methods are simple, but they become slow and unreliable when the number of students and classes increases. A teacher may spend several minutes marking attendance, students may be marked incorrectly, and the attendance percentage may not be visible to students until it is too late to recover.

The Smart Attendance Management System is designed as a modern solution to this problem. It combines a web application, a backend API, a database and an artificial intelligence service. Teachers can create classes, add or invite students, run attendance using classroom photographs, verify results, manage students, schedule exams and review leave requests. Students can join classes, enroll their face profile, view attendance analytics, upload leave proof, check exam eligibility and receive notifications. The system is not only an attendance marker; it is an academic support platform that explains what the attendance data means.

The product name used in the user interface is MarkIn / SAMS. SAMS stands for Smart Attendance Management System. The word MarkIn represents the idea of marking attendance in a smarter and safer way. The project is implemented as a three-service architecture: React frontend, Express backend and FastAPI AI service. MongoDB is used for persistent storage, and the AI service uses face detection and recognition models to support photo-based attendance.

1.2 Motivation

The main motivation behind this project is to make attendance more accurate, transparent and useful. Traditional attendance systems usually solve only one task: recording present or absent. In practice, an institution needs much more. Teachers need a clean roster and a safe

way to correct AI mistakes. Students need to know which class is weak, how many classes they must attend before an exam and whether leave proof was approved. Administrators need records that can be audited and exported.

The project therefore focuses on four ideas:

1. **Reduce teacher workload:** Camera-based attendance, QR attendance and manual controls reduce repetitive work.
2. **Protect correctness:** AI suggestions are not blindly finalized; a teacher reviews Today Attendance Data before submission.
3. **Inform students early:** Low attendance, absence, leave status and exam eligibility are communicated through dashboards and email.
4. **Keep data useful:** Attendance is converted into analytics, recovery guidance, class summaries and CSV reports.

1.3 Problem Statement

Educational institutions require a reliable attendance management system that can record attendance efficiently, reduce manual errors, support verification, notify students about important attendance events and provide meaningful analytics. Existing manual or semi-digital methods are often slow, fragmented and reactive. They do not provide real-time correction workflows, AI-assisted classroom verification, leave proof handling, exam eligibility planning or email-based communication in one integrated product.

The problem can be stated as follows:

To design and implement a full-stack Smart Attendance Management System that allows teachers and students to manage attendance, class activities, AI-assisted verification, leave documentation, exam eligibility and notifications through a secure and user-friendly web platform.

1.4 Objectives

The objectives of the project are:

- To provide separate student and teacher dashboards with role-based access.
- To allow teachers to create classes and students to join using class codes.
- To allow students to enroll face profiles for classroom photo attendance.
- To implement AI-assisted recognition using a separate FastAPI service.

- To create a review-first attendance workflow where teachers verify results before final submission.
- To send email notifications for verification, password reset, absence, leave status, exam warnings and class cancellation.
- To provide student analytics such as total classes, present count, absent count, percentage and recovery guidance.
- To support leave or medical proof upload with teacher approval/rejection.
- To support exam scheduling and attendance eligibility calculation.
- To keep the architecture modular, maintainable and scalable.

1.5 Scope of the Project

The current project scope includes frontend user experience, backend business rules, MongoDB persistence, AI-service integration and email notification workflows. The system supports real classroom workflows such as class creation, joining, attendance capture, draft review, final submission, QR attendance, manual attendance, leave proof, assignments, discussion, notes, exams and analytics.

The project does not claim to replace institutional ERP systems completely. Instead, it focuses on attendance-centered academic operations. It can later integrate with ERP, biometric hardware, institutional SSO, payment systems or official exam portals if required.

1.6 Users of the System

Table 1.1: Primary users of the system

User Type	Purpose in System
Student	Joins classes, enrolls face profile, views attendance, checks exam eligibility, receives notifications and submits leave proof.
Teacher	Creates classes, manages rosters, runs attendance, corrects Today Attendance Data, finalizes attendance, reviews leave requests and schedules exams.
AI Service	Performs face detection, embedding generation, roster matching and attendance session analysis.
Backend Service	Owns authentication, authorization, MongoDB access, email delivery, attendance rules and API orchestration.

1.7 Why the Project is Useful

The usefulness of the project is visible from both student and teacher perspectives. A student does not need to guess whether attendance is safe for exams. The dashboard shows joined classes, attendance percentages, present/absent counts, upcoming exams and minimum classes to attend. A teacher does not need to manually call every student name when a clear classroom photo is available. The AI service suggests attendance, but final control remains with the teacher.

The system also solves a common communication gap. If a student is marked absent from photo attendance, the student receives an email immediately after the draft is created. If the student believes the result is wrong, the teacher can correct the draft before final submission. This small workflow makes the system more trustworthy than a direct AI auto-submit model.

1.8 Report Organization

This report is organized to help a beginner understand the project step by step. Chapter 2 discusses existing systems and related concepts. Chapter 3 explains requirements. Chapter 4 describes architecture and design. Chapter 5 explains implementation. Chapter 6 explains features and working flows. Chapter 7 presents testing and results. Chapter 8 concludes the work with limitations and future scope. The appendix contains route summaries, setup guidance and additional reference material.

Chapter 2

Literature Review and Existing Systems

2.1 Background

Attendance management has evolved from paper registers to digital portals and biometric systems. The earliest method is the manual register, where the teacher calls student names and marks presence. Later, spreadsheet-based records and ERP portals reduced paper usage but still required manual entry. Biometric devices such as fingerprint scanners improved automation but require fixed hardware and may not be suitable for every classroom. QR-based systems are easy to deploy but can be misused if students share codes.

Recent systems use face recognition to identify students from images or video. This approach can reduce teacher workload, but it must be designed carefully. Classroom photographs may contain partial faces, different angles, low lighting, occlusions and multiple students. An attendance system should therefore avoid blind automatic finalization. The safest approach is assisted intelligence: AI suggests, teacher verifies and the final record is written only after review.

2.2 Manual Attendance Systems

Manual systems remain common because they are easy to understand and require no technical infrastructure. However, they are time-consuming and error-prone. A teacher may accidentally mark a student absent, and students may not learn about low attendance until exam time. Manual systems also make reporting difficult because the data is scattered across notebooks or spreadsheets.

Table 2.1: Limitations of manual attendance

Limitation	Impact
Time consumption	Classroom teaching time is reduced because attendance must be called manually.
Human error	Wrong marking can occur due to missed names, late entries or register mistakes.
Delayed visibility	Students may not know exact attendance percentage until records are calculated.
Weak reporting	Manual data is difficult to analyze, export or connect with exam eligibility rules.
No automatic notification	Students and parents are not informed immediately about absence or risk.

2.3 Digital ERP Attendance Systems

Many institutions use ERP portals where teachers enter attendance into a web form. Such systems improve record storage and calculation but often do not reduce the teacher's marking effort. They may also be generic and not optimized for classroom realities such as face enrollment, leave proof, AI review, student-side recovery planning or exam eligibility.

The proposed system keeps the useful parts of ERP systems, such as persistent records and role-based access, but adds attendance-specific workflows. For example, the teacher class workspace includes AI photo attendance, Today Attendance Data, manual attendance, QR attendance, CSV download, student details, leave requests and exam setup.

2.4 Biometric Attendance Systems

Fingerprint or RFID-based attendance systems are common in offices and some institutions. They can verify presence using hardware devices. However, classroom use may face practical problems: device cost, queues, hygiene concerns, hardware maintenance, and lack of flexibility when classes move between rooms. Face recognition from classroom photos is more flexible because it can use a teacher's phone or uploaded images.

At the same time, face recognition is not perfect. It must be combined with quality checks, confidence scores and teacher review. SAMS therefore treats AI recognition as a decision-support component, not a final authority.

2.5 QR Attendance Systems

QR attendance is popular because it is simple. A teacher displays a QR code, and students scan it to mark themselves present. The main advantage is speed. The main risk is misuse: a student can share the QR link with someone outside the room unless extra controls are used. In SAMS, QR attendance is included as an alternative attendance method, but AI photo attendance and manual teacher control remain available.

2.6 Face Recognition Concepts

Face recognition systems generally involve four steps:

1. Detect faces in an image.
2. Align or normalize detected faces.
3. Convert each face into a numerical embedding.
4. Compare embeddings with stored face profiles.

The AI service in this project uses InsightFace model packs. Detection is performed by models such as SCRFD or RetinaFace, and recognition uses ArcFace-style embeddings. A face embedding is a vector representation of a face. If two embeddings are close by cosine similarity, they are likely to belong to the same person. The system also compares against the second-best match to avoid accepting ambiguous results.

2.7 Related Technology Stack

The project uses a MERN-style web architecture combined with a Python AI service. This split is useful because web business rules and machine learning workloads have different needs. React is suitable for interactive dashboards. Express is suitable for REST APIs and orchestration. MongoDB is suitable for flexible document storage. FastAPI is suitable for model-serving endpoints and Python ML libraries.

Table 2.2: Technology stack overview

Layer	Technology	Reason
Frontend	React, Parcel, JavaScript	Component-based dashboards, fast development and single-page routing.
Backend	Node.js, Express	REST API, authentication, classroom rules and email orchestration.
Database	MongoDB, Mongoose	Flexible documents for classes, users, records, leave requests and email logs.
AI Service	FastAPI, Python	Model serving, image processing and machine learning workflows.
Face Models	InsightFace, ArcFace, SCRFD	Real face detection and recognition model support.
Email	Nodemailer, Gmail SMTP	OTP, password reset, absence and academic notifications.

2.8 Gap in Existing Solutions

Most attendance systems focus on marking presence. Fewer systems combine attendance marking with student learning support. The important gap is not just recording who was absent, but helping the student act on it. SAMS addresses this gap with:

- student analytics and per-class detail pages;
- exam eligibility warnings and calculator;
- leave/medical proof upload;
- email alerts and notification center;
- teacher-controlled AI attendance draft review;
- class notes, assignments and discussion.

2.9 Proposed Direction

The proposed direction is a review-first smart attendance platform. AI is used to speed up attendance recognition, but the teacher remains responsible for final submission. This balances automation and accountability. The system also turns attendance data into useful academic guidance rather than treating it as a hidden administrative record.

Chapter 3

Requirement Analysis

Requirement analysis explains what the system must do, who will use it, which data it must handle, and what quality conditions must be satisfied. For MarkIn / SAMS, the requirements were derived from the practical problems faced in daily attendance work: teachers need faster marking, students need transparency, and administrators need dependable records. The system therefore combines attendance capture, verification, analytics, document proof, exam eligibility and email communication into one connected workflow.

3.1 Stakeholders

The project has two primary user groups and a few supporting stakeholders. A teacher operates classes, attendance sessions, leave approvals and exam rules. A student joins classes, checks attendance, submits documents and receives alerts. The department or institute can use the stored data for records, compliance and reports. Parents may optionally receive absence alerts when the student profile contains a parent email address.

Table 3.1: Stakeholders and their expectations

Stakeholder	Main expectation	How SAMS supports it
Student	Clear attendance status, class-wise records, exam eligibility, proof submission and alerts.	Student dashboard, class detail pages, attendance calculator, leave proof upload, notification page and email alerts.
Teacher	Fast attendance capture, reliable correction, class management and student analytics.	Teacher dashboard, AI photo attendance, Today Attendance Data draft, manual edits, QR attendance, leave review and exam setup.
Department	Consistent attendance records and reduced manual paperwork.	MongoDB-backed records, final attendance submission, export-ready summaries and audit-friendly email logs.
Parent or guardian	Optional awareness when a student is absent.	Optional parent email on the student profile for absent notifications.
System maintainer	Modular codebase and simple deployment.	Separate frontend, backend and AI service with environment-based configuration.

3.2 Functional Requirements

Functional requirements describe the visible actions supported by the application. The first major requirement is authentication. Students and teachers should be able to register, verify their email using an OTP, log in, update profile details and reset forgotten passwords. The system must not treat an unverified account as fully active because email is used for important academic notifications.

The second requirement is class management. A teacher should be able to create a class with subject name, subject code, section, semester, academic year, room and weekly schedule. Students should be able to join a class through a join code or join link. The teacher should be able to view the roster and add, update or remove students when needed.

The third requirement is attendance capture. The system should allow manual attendance, QR attendance and AI-assisted photo attendance. Manual attendance is useful when a teacher wants direct control. QR attendance is useful when students physically present in class can scan a session code. AI-assisted photo attendance is useful when the teacher wants to take classroom photos and let the system identify likely present students.

The fourth requirement is review before finalization. Photo attendance cannot be considered perfect because classroom lighting, camera angle, occlusion and missing face enrollment can affect recognition. Therefore, AI results must first go to a draft area called Today Attendance Data. The teacher can add or remove students from this draft. Absentees are notified early

so that a mistaken absence can be reported before final submission. If the teacher does not finalize manually, the backend scheduler can finalize eligible drafts after the configured delay.

The fifth requirement is student usefulness. A student dashboard should not simply show a percentage. It should show total classes, present count, absent count, class-wise status, upcoming exams, safe attendance information, notifications and navigation to separate class pages. Each class page should provide schedule, recent attendance, leave proof submission and useful context for improving attendance.

The sixth requirement is academic communication. The backend must send professional emails for signup verification, welcome message, password reset, profile email verification, leave approval or rejection, absent notification, exam attendance warning and class cancellation or rescheduling. The email module should be modular, retry failed delivery and store delivery status without exposing SMTP secrets.

Table 3.2: Functional requirement summary

Module	Requirement	Expected behaviour
Authentication	Email verified registration	User enters registration details, receives OTP, verifies email and then receives a welcome email.
Authentication	Forgot password	User requests reset, receives OTP or reset token, and can set a new password before expiry.
Profile	Email update	User submits new email, verifies OTP and only then the profile email is changed.
Classroom	Teacher creates class	Teacher defines subject, code, section, schedule and metadata. System generates join code.
Classroom	Student joins class	Student submits join code and becomes part of the class roster.
Attendance	Manual marking	Teacher can directly mark present, absent, late, excused or cancelled records.
Attendance	QR marking	Teacher creates a QR session; students scan to mark attendance within the allowed window.

Module	Requirement	Expected behaviour
Attendance	Photo marking	Teacher uploads classroom photo; backend sends it to AI service; suggested attendance is stored as a draft.
Draft review	Today Attendance Data	Teacher sees present and absent names, corrects the draft and finalizes it.
Leave proof	Student upload	Student submits reason, absence date and image or PDF proof from the class detail page.
Leave review	Teacher decision	Teacher previews files online, approves or rejects, and student receives email.
Exam planning	Teacher exam setup	Teacher sets exam date and required attendance percentage.
Exam planning	Student eligibility	Student sees eligibility, minimum classes to attend and warnings when below threshold.
Notifications	Dashboard alerts	Important alerts appear under the student notification bell and notification page.
Email	Delivery logging	Each email event is queued, retried and written to delivery logs with safe metadata.

3.3 Non-Functional Requirements

Non-functional requirements describe the quality of the system rather than a single button or page. In an attendance system these qualities are important because incorrect records can directly affect student eligibility. The system must be reliable, understandable, responsive, secure and maintainable.

Table 3.3: Non-functional requirements

Quality attribute	Requirement for this project
Usability	Dashboards should show the most useful information first and should avoid confusing students with raw data only.
Accuracy	AI attendance should be treated as an assistive result, not an unquestioned final record. Teacher review remains part of the workflow.
Security	Passwords, email OTPs, SMTP credentials and database connection strings must not be exposed in frontend code or reports.
Performance	Dashboard APIs should return summarized data so that the frontend can render quickly without doing heavy calculations.
Scalability	Email sending should use a queue and limited concurrency so bulk class notifications do not block normal API responses.
Maintainability	Frontend, backend and AI logic should remain separated so each service can evolve independently.
Observability	Health and email status endpoints should show safe operational state without revealing secrets.
Portability	The app should run locally through workspace scripts and should support Docker-based deployment.

3.4 User Roles and Permissions

Role separation is necessary because students and teachers perform different actions. The student should not be able to finalize attendance or approve leave. The teacher should not need direct database access. The backend API is the control point for enforcing these actions.

Table 3.4: User roles and permissions

Action	Student	Teacher
Register and verify email	Allowed	Allowed
Log in and update profile	Allowed	Allowed
Create class	Not allowed	Allowed
Join class	Allowed	Not required
View own attendance	Allowed	Can view class attendance
View all students in a class	Not allowed	Allowed for own classes
Submit leave proof	Allowed for own class membership	Not applicable
Preview leave proof	Own submissions only	Allowed for students in own class
Approve or reject leave	Not allowed	Allowed
Set exam date and eligibility threshold	Not allowed	Allowed
Use attendance calculator	Allowed	Can interpret class-level analytics
Create attendance draft	Not allowed	Allowed
Edit and finalize Today Attendance Data	Not allowed	Allowed

3.5 Use Case View

A beginner can understand the system through use cases. A use case is a user goal written from the user's perspective. For example, "student checks exam eligibility" is a use case. It is not just a page; it includes input data, processing and result.

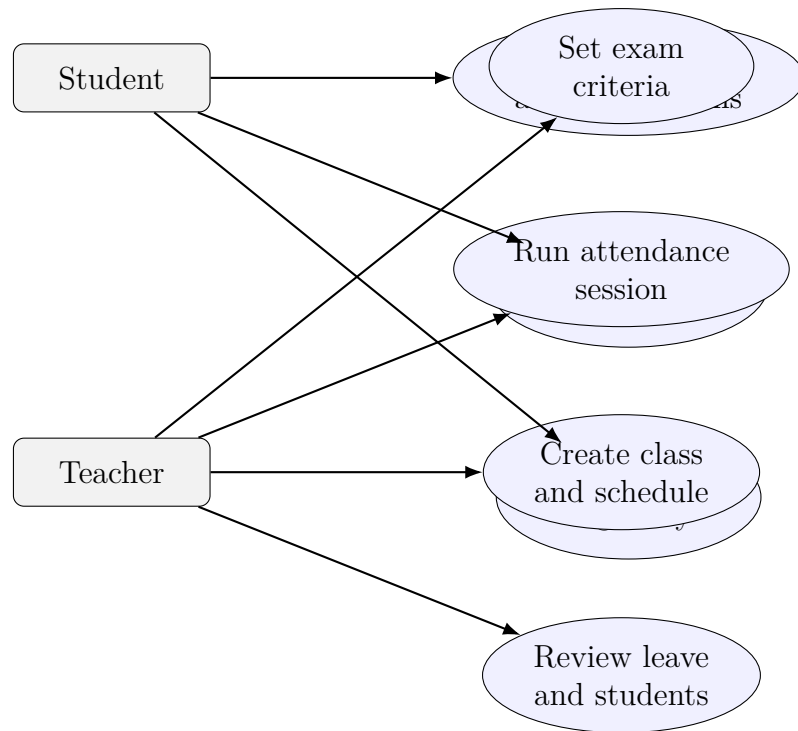


Figure 3.1: High-level user role use cases

3.6 Data Requirements

The system requires structured data about users, classes, attendance, face enrollment, leave requests, exams and email events. MongoDB is a suitable choice because class and attendance documents may contain nested schedule slots, attachment metadata and draft records. The backend still defines schemas using Mongoose so the application has validation and consistent field names.

Table 3.5: Important data required by SAMS

Data item	Purpose
User profile	Stores role, user id, name, email, verification state, optional parent email and profile metadata.
Classroom	Stores teacher ownership, subject information, section, room, schedule and join code.
Class membership	Connects a student to a class and supports roster analytics.
Face profile	Stores enrollment state and face embeddings or metadata needed by the AI-backed workflow.
Attendance record	Stores final attendance decisions with status, date, class and source.
Attendance draft	Stores temporary Today Attendance Data before final submission.
Leave request	Stores absence date, reason, request type, attachments and review decision.
Class exam	Stores exam date, required attendance percentage and teacher notes.
Email delivery log	Stores delivery status, template name, recipient metadata and error details if sending fails.

3.7 Input and Output Requirements

Inputs must be simple enough for a normal college user. A teacher should only need class details, attendance photos, QR controls or manual status selections. A student should only need join code, profile details, absence reason, proof files and optional calculator inputs. The backend should validate these inputs before storing them.

The outputs should be actionable. Instead of only showing raw attendance records, the student dashboard should answer questions such as: How many classes have I joined? Which class is risky? How many total sessions were held? How many were present or absent? Am I eligible for the next exam? How many future classes should I attend to be safe? The teacher dashboard should answer questions such as: Which classes need action today? Which students are below range? Which leave requests are pending? Which attendance drafts still require final submission?

3.8 Constraints and Assumptions

The project assumes a college environment where classes are organized by teacher, subject and section. The system assumes that students and teachers have email access because verification and alerts are part of the workflow. It also assumes that the teacher remains responsible for final attendance accuracy because computer vision cannot be perfect under all classroom conditions.

The major technical constraints are camera quality, lighting conditions, network availability and the availability of AI model files. If the AI service is unavailable, manual or QR attendance

can still support the academic workflow. If email delivery fails, the backend records the failure and can be configured with retries. If a student has not completed face enrollment, the system warns the student and the teacher can rely on other attendance methods.

3.9 Requirement Traceability

Requirement traceability helps connect features to project modules. This makes testing easier because each requirement can be mapped to a backend service and a frontend page.

Table 3.6: Requirement traceability matrix

Requirement	Backend area	Frontend area
Email verification	Auth and email OTP services	Signup and verification screens
Student dashboard analytics	Student dashboard service and attendance analytics service	Student dashboard, notification page and class detail page
Teacher dashboard analytics	Teacher dashboard service and classroom services	Teacher dashboard and class students page
Face enrollment	Face profile service and AI service bridge	Student face enrollment page
Photo attendance	Attendance service, teacher classroom controller and AI gateway	Teacher attendance workspace
Today Attendance Data	Attendance draft model, scheduler and finalize endpoints	Teacher draft review section
Leave proof	Leave request service, attachment metadata and email notification service	Student class detail and teacher student detail page
Exam eligibility	Exam schedule service and attendance analytics	Upcoming exams section and calculator page
Notifications	Dashboard services and student notification builder	Bell icon and separate notification view

Requirement	Backend area	Frontend area
Email status	Email service and delivery logs	Email setup or test/status view

This chapter defined the expectations of the system. The next chapter converts these requirements into an architecture and design that explains how the frontend, backend, database and AI service work together.

Chapter 4

System Design and Architecture

System design explains how the requirements are divided into cooperating parts. MarkIn / SAMS follows a three-service architecture: a React frontend for user interaction, an Express backend for business rules and persistence, and a FastAPI AI service for face recognition. MongoDB stores the application data. This separation keeps the browser simple, keeps security-sensitive work inside the backend, and keeps machine learning code independent from normal web application code.

4.1 Architectural Principles

The design follows five practical principles. First, the backend is the source of truth. The frontend can display data and collect input, but it should not decide final attendance rules or directly contact the AI service. Second, AI should assist rather than replace teacher review. Third, every important academic event should be traceable: attendance finalization, leave approval, email delivery and exam eligibility are represented in backend records. Fourth, sensitive configuration must stay in environment variables and must never be included in source reports. Fifth, dashboards should present summarized information because students and teachers need decisions, not only raw tables.

4.2 Overall Architecture

Figure 4.1 shows the system architecture used by the project. The browser communicates only with the backend REST API. The backend validates requests, reads or writes MongoDB documents, sends email through SMTP and forwards recognition jobs to the AI service. The AI service loads face detection and recognition models and returns structured results.

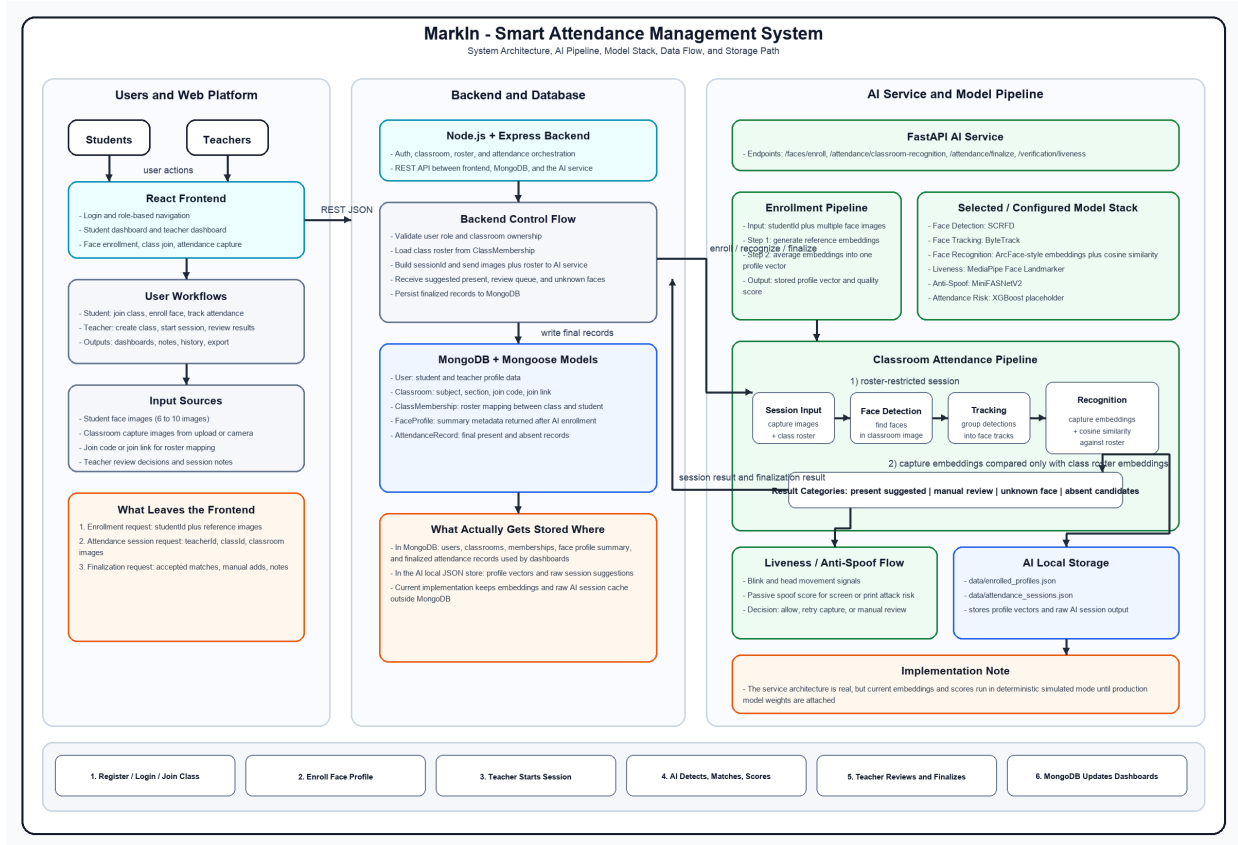


Figure 4.1: Overall MarkIn/SAMS architecture and workflow

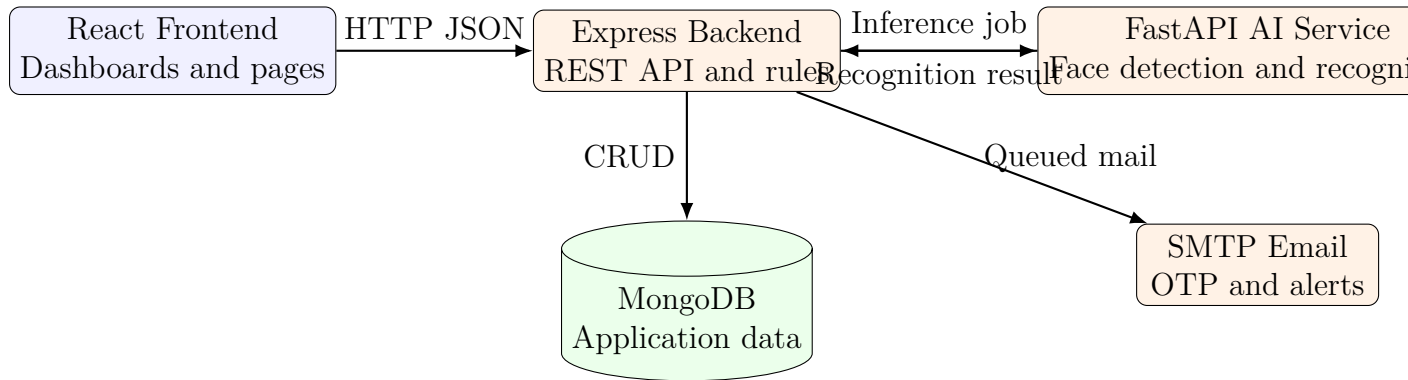


Figure 4.2: Service-level communication

4.3 Frontend Design

The frontend is built with React and JavaScript. It contains separate pages for the home page, authentication, student dashboard, student tools, student class detail, face enrollment, QR attendance, teacher dashboard, teacher class creation, teacher profile and teacher classroom students. The frontend service layer in `frontend/src/services/api.js` centralizes API calls so that pages do not duplicate endpoint strings and error handling.

The frontend design is role-oriented. After login, a student sees attendance health, classes joined, upcoming exams, notification count and tools such as the attendance calculator. The class detail page gives a separate view for each class, including attendance history, schedule and leave proof form. A teacher sees classes, schedule, student lists, attendance controls, Today Attendance Data and leave reviews.

4.4 Backend Design

The backend is built with Express and Mongoose. It exposes REST endpoints under the API router. Controllers receive requests and services contain reusable business logic. Models define MongoDB schemas. This separation allows the project to grow without placing all logic inside routes.

Table 4.1: Backend layer responsibilities

Layer	Examples	Responsibility
Routes	<code>backend/src/routes/index.js</code>	Maps URL paths and HTTP verbs to controllers.
Controllers	Auth, student, teacher, classroom and attendance controllers	Validate request shape, call services and return JSON responses.
Services	Email, attendance, dashboard, leave, exam and AI gateway services	Implement business rules, calculations and integrations.
Models	User, Classroom, AttendanceRecord, AttendanceDraft and others	Define persistent document structure and indexes.
Configuration	Environment and database configuration	Loads safe runtime settings and connects to MongoDB.

4.5 AI Service Design

The AI service is separated from the backend because face recognition has different dependencies and lifecycle needs. It is written with FastAPI and contains routes, services, pipelines, storage helpers and schemas. The relevant pipelines include enrollment, live verification and classroom attendance. The system can use InsightFace model packs such as Antelopev2 or Buffalo_L, depending on available local assets.

In the attendance workflow, the backend sends classroom images and enrolled face profiles to the AI service. The AI service detects faces, extracts embeddings, compares them with enrolled student profiles and returns a result with matched students, unmatched faces and confidence notes. The teacher sees useful verification notes rather than every raw internal score.

4.6 Database Design

MongoDB stores the state of the system as collections. A simplified entity relationship view is shown in Figure 4.3. Although MongoDB is document-oriented, the application still has relationships through ids such as `classId`, `studentUserId` and `teacherUserId`.

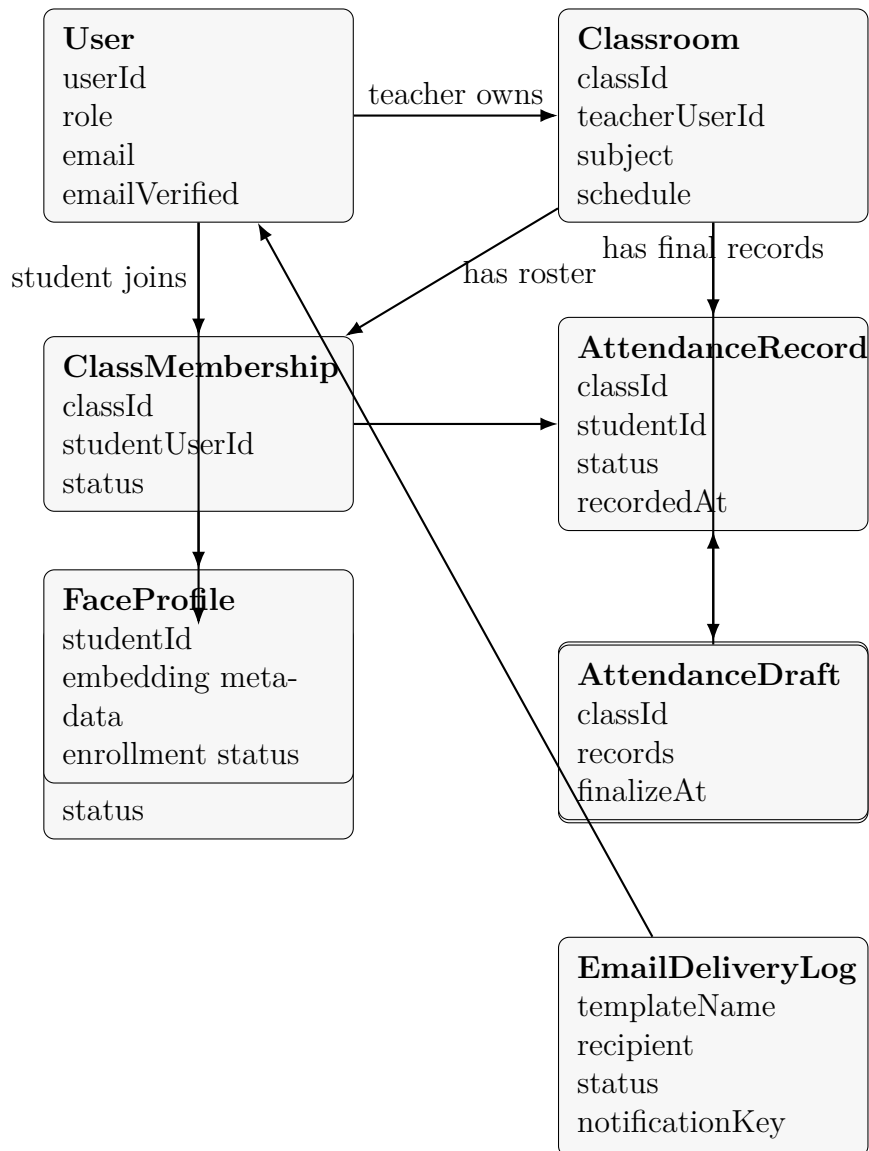


Figure 4.3: Simplified database/entity relationship diagram

Table 4.2: Database collection summary

Collection	Purpose
users	Stores student and teacher accounts, profile details and email verification state.
classrooms	Stores class identity, teacher ownership, schedule, room and join code.
classmemberships	Stores student membership in classes.
attendancerecords	Stores finalized attendance entries used for analytics.
attendancedrafts	Stores Today Attendance Data before final submission.
faceprofiles	Stores face enrollment metadata for student recognition.
leaverequests	Stores medical or leave proof, attachments and teacher review state.
classexams	Stores exam dates and required attendance percentages.
emailotps	Stores temporary OTP entries with expiry and purpose.
emaildeliverylogs	Stores email delivery attempts, status and errors.
qrattendancesessions	Stores QR session windows and class attendance scan state.
classassignments	Stores assignments created by teachers.
classassignmentssubmissions	Stores student assignment submissions.
classdiscussionmessages	Stores class discussion posts, replies and reactions.

4.7 Today Attendance Data Design

The Today Attendance Data feature is one of the most important project workflows. The design avoids a common weakness of automatic attendance systems: immediately writing AI output as final attendance. Instead, the backend creates a draft. The draft has a list of present and absent students, recognition notes, source photos, timestamps and finalization status. The draft remains editable for the teacher. The system can send absence notifications as soon as the draft is created, but final records are written only when the teacher finalizes or the automatic finalization period expires.

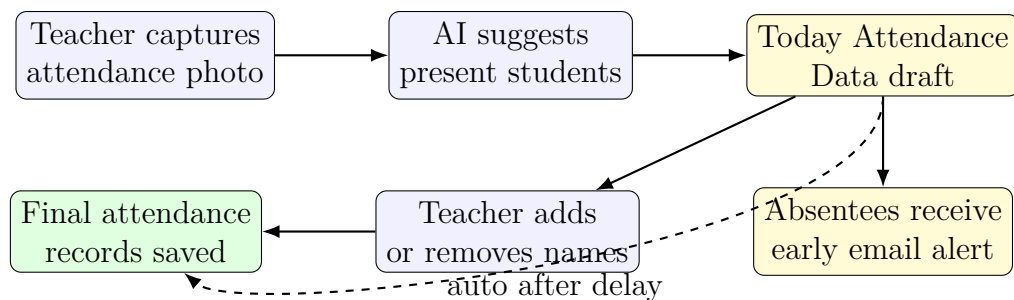


Figure 4.4: Today Attendance Data draft and finalization flow

4.8 Email Notification Architecture

Email is handled by a dedicated backend module using Nodemailer. The service is configurable through environment variables. Email sending is disabled by default in development unless explicitly enabled. When enabled, the service verifies SMTP connectivity, uses professional templates and records delivery status. The email module does not print credentials in status responses.

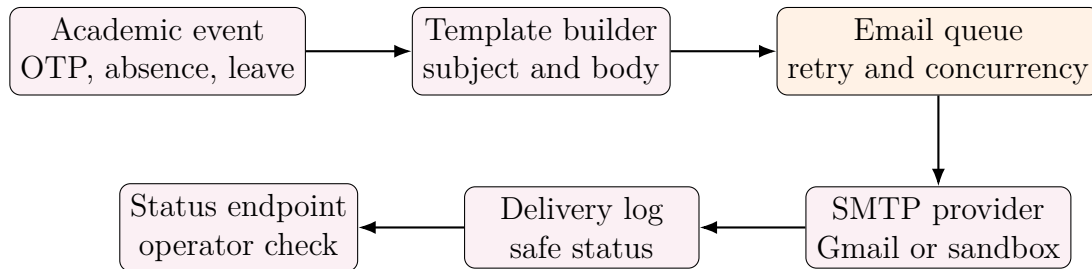


Figure 4.5: Email notification flow

4.9 Security Design

The most important security decisions are simple and practical. Credentials stay in backend environment variables. The frontend never receives SMTP password, database URL or AI model paths. The email status endpoint returns only safe information such as enabled state, host name, sender and verification status. OTP records have an expiry time. Profile email updates require OTP verification. Leave attachments are shown through controlled application data rather than requiring teachers to download files blindly.

For a production version, this design should be extended with stronger session management, role-based middleware, rate limits for OTP requests, file size restrictions, antivirus scanning for uploaded documents and more detailed audit logs. The current design already keeps the main sensitive values away from public pages and report content.

4.10 API Design Style

The REST API uses resource-oriented routes. For example, student dashboard data is requested using a student id, teacher class actions use teacher id and class id, and leave review uses request id. This makes URLs readable and helps map actions to the user role.

Table 4.3: Representative API design examples

Method	Path	Purpose
GET	/api/v1/students/:userId/dashboard	Fetch student analytics, classes, notifications and exam information.
POST	/api/v1/students/:studentId/classes/join	Join a teacher-provided join code.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session	Create a Today Attendance Data draft.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/attendance/today-draft/:draftId	Update Today Attendance Data draft before final submission.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/today-draft/:draftId/submit	Submit Today Attendance Data draft.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/exam	Update exam and required attendance percentage.

This chapter explained how the system is arranged. The next chapter explains how these designs are implemented in code.

Chapter 5

Implementation Details

This chapter explains how the design is converted into working software. The project is implemented as a monorepo with three application services. The frontend handles browser pages, the backend handles API and business rules, and the AI service handles face recognition pipelines. The implementation is intentionally modular so that one part can be improved without rewriting the entire application.

5.1 Project Structure

The top-level project structure keeps related code together. A beginner reading the repository can first identify the service and then inspect the pages, controllers, services or models inside it.

```
SAMS/  
  frontend/      React and Parcel user interface  
  backend/       Express REST API, Mongoose models and services  
  ai-service/    FastAPI face recognition service  
  docs/          Architecture, roadmap and email notes  
  reports/       Generated LaTeX report workspace  
  docker-compose.yml
```

The backend also contains a service-oriented structure:

```
backend/src/  
  controllers/   Request handlers for API routes  
  models/        Mongoose schemas  
  routes/        Central API router  
  services/      Business logic and integrations  
  config/        Environment and database setup
```

5.2 Technology Stack

The chosen technologies are common enough for student developers to understand and powerful enough to support the required features. Table 5.1 summarizes the stack.

Table 5.1: Technology stack used in implementation

Layer	Technology	Reason for use
Frontend	React, JavaScript, Parcel	Component-based UI with fast local development and reusable dashboard sections.
Backend	Node.js, Express	Simple REST API framework suitable for authentication, dashboards and attendance workflows.
Database	MongoDB, Mongoose	Flexible document storage with schemas for users, classes, records, leave requests and drafts.
AI service	Python, FastAPI	Clean API layer for face recognition pipelines and model readiness endpoints.
Face recognition	InsightFace model packs	Provides practical face detection and embedding-based recognition capability.
Email	Nodemailer, SMTP	Supports Gmail app passwords or free SMTP sandbox services.
Development	npm workspaces, Docker	Allows local multi-service development and container-based deployment.
Documentation	LaTeX	Produces a formal, editable and printable academic report.

5.3 Backend Implementation

The backend starts with configuration, database connection and route registration. API routes are grouped inside `backend/src/routes/index.js`. This single router maps paths to controllers. Controllers then call services. For example, authentication routes call authentication and OTP services, while teacher classroom routes call attendance, classroom, leave and exam services.

The service layer is the most important backend implementation decision. It prevents controllers from becoming too large. For example, `studentDashboardService.js` prepares student dashboard data, `attendanceAnalyticsService.js` summarizes records, `emailNotificationService.js` chooses notification templates, and `attendanceDraftScheduler.js` handles automatic draft finalization.

5.3.1 Authentication and Email Verification

During signup, the backend creates the user in a verification-required state and sends an OTP email. The user becomes active only after the OTP is verified. This prevents invalid or mistyped email addresses from receiving future academic alerts. Password reset also uses a secure temporary OTP flow. Profile email update uses a separate OTP purpose so that changing an email address cannot happen silently.

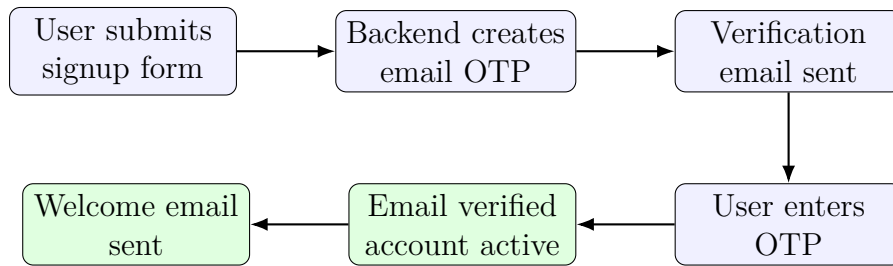


Figure 5.1: Email verified registration implementation flow

5.3.2 Classroom and Membership Implementation

The classroom module stores the teacher id, class details, schedule slots and join code. A class schedule slot includes weekday and time information. When a student joins a class, the backend creates a membership document. Attendance analytics later use the membership list to know which students belong to a class and which students must appear in class-level calculations.

The join-code design is simple and practical. A teacher can share a code or link. A student can join without the teacher manually typing every name. At the same time, the teacher can still add, update or remove students from the roster page, which is important when data is incomplete.

5.3.3 Attendance Implementation

The attendance implementation supports several sources. Manual attendance is direct. QR attendance uses a teacher-created session and a student scan endpoint. AI photo attendance sends image data to the AI service and produces a draft. All final records are stored in the attendance record collection with status and source fields.

Table 5.2: Attendance status and meaning

Status	Meaning	Typical source
Present	Student attended the class.	Manual, QR or AI-assisted session.
Absent	Student did not attend or was not confirmed present.	Draft finalization or manual entry.
Late	Student attended after expected time.	Manual teacher decision.
Excused	Student's absence is accepted through proof or teacher decision.	Leave review or manual decision.
Class cancelled	Class did not happen and should not harm attendance.	Teacher cancellation workflow.

5.3.4 Today Attendance Draft Implementation

When AI-assisted attendance is run, the backend does not immediately create final attendance records. It creates an **AttendanceDraft**. The draft includes class id, teacher id, recognized records, suggested present list, suggested absent list, verification notes and a finalization time. The teacher can update this draft using the draft update endpoint. Finalization converts the draft to attendance records.

This design supports a useful human-in-the-loop workflow. Suppose a student was present but hidden behind another student in the photo. The AI result might mark that student absent. The absent email gives the student a chance to tell the teacher. The teacher can then add the student in Today Attendance Data before final submission. This keeps the system useful without pretending that computer vision is perfect.

5.4 Student Dashboard Implementation

The student dashboard is implemented as a summary page supported by separate pages for tools and class details. The dashboard service computes totals, percentages, attendance trend, joined classes, upcoming exams and warnings. The frontend uses that data to show helpful cards and links.

Table 5.3: Student dashboard data shown to users

Data item	Student value
Classes joined	Shows how many active academic classes the student belongs to.
Total sessions held	Shows total attendance opportunities already recorded.
Present and absent counts	Converts percentage into understandable numbers.
Attendance percentage	Helps student know current standing.
Class-wise cards	Allows student to open a separate detail page for every class.
Upcoming exams	Connects attendance percentage to exam eligibility.
Minimum classes to attend	Shows a clear action target rather than only a warning.
Notification count	Alerts the student about risky classes, exams and leave decisions.
Face enrollment status	Reminds student to complete enrollment for photo attendance.

5.5 Teacher Dashboard Implementation

The teacher dashboard focuses on action. A teacher can create classes, see schedule, open class management, run attendance, create assignments, set exam rules and review leave requests. The teacher class students page provides roster analytics, low-attendance filters and a detailed student section. This is where teachers can preview leave proof files such as PDF or images online and approve or reject them.

Teacher actions are scoped by teacher id and class id. This makes it possible for the backend to verify that a teacher is operating on their own class before changing attendance, roster or exam data.

5.6 Leave and Medical Proof Implementation

The leave request feature is implemented in the student class detail page and teacher classroom students page. A student selects a request type, absence date, reason and attachment files. Files are converted to safe attachment metadata that the backend stores with the request. The teacher can open the student detail section and preview attachments. Images are shown inline and PDFs are shown in a browser preview area. The teacher can approve or reject with an optional note.

When the decision is saved, the email notification module sends a leave review email. The email contains the class, absence date and approval status. This helps students know the decision without repeatedly checking the dashboard.

5.7 Exam Schedule and Eligibility Implementation

The teacher can set an exam date and required attendance percentage for a class. The backend stores this in the class exam model. The student dashboard reads upcoming exams and calculates eligibility using current attendance and scheduled classes before the exam. The calculation can explain whether the student is currently safe, close to risk or below requirement.

The attendance calculator allows the student to enter an exam date and required percentage. The system can use the class schedule to estimate how many classes may be held before that exam date. The output is the minimum number of classes the student should attend to be safe. The general formula is:

$$\text{required presents} = \left\lceil \frac{p}{100} \times \text{projected total classes} \right\rceil$$

$$\text{minimum future attendance} = \max(0, \text{required presents} - \text{current presents})$$

Here p is the required attendance percentage. If the minimum future attendance is greater than the number of classes expected before the exam, then the student cannot mathematically reach the required percentage by only attending future classes. The dashboard can then show a stronger warning.

5.8 Email Service Implementation

The email system is implemented as a modular notification layer. It contains template builders, a sending service, OTP service and notification service. The environment controls whether email is enabled, which SMTP host is used, how many retries are allowed and whether OTPs can be exposed during development. In real operation, OTPs are not returned in API responses.

Table 5.4: Email notification events implemented in the backend

Event	Trigger	Email content
Signup verification	New student or teacher registration	OTP, expiry and verification instructions.
Welcome message	Successful email verification	Student or teacher name, login email and account activation confirmation.
Password reset	User requests recovery	OTP or secure reset instructions and expiry warning.

Event	Trigger	Email content
Profile email	User updates profile	OTP for confirming the new email address.
Leave decision	Teacher approves or rejects a request	Class, leave date, status and teacher note when available.
Exam warning	Attendance below exam criterion	Current percentage, required percentage and action needed.
Absent notice	Student marked absent in draft or record	Class, date, subject and timing context.
Class cancellation	Teacher cancels or reschedules	Updated class information and schedule note.

5.9 AI Attendance Implementation

The AI service contains three main pipeline areas: enrollment, live verification and classroom attendance. During enrollment, the student's face profile is captured and stored. During classroom attendance, the service detects faces in the submitted image, creates embeddings and compares them with enrolled profiles. The output is a list of matched students and useful notes.

The backend treats the AI service as a dependency that may be healthy, ready or degraded. Health endpoints help show whether MongoDB and AI runtime are available. If the AI model pack is missing or the service is still starting, the app can show degraded state instead of silently failing.

5.10 Frontend Navigation Implementation

The frontend uses hash-based routing. This makes the app easy to run in local development without server-side route rewriting. Pages are organized around real user tasks:

- Student dashboard for overview.
- Student class detail page for a single class.
- Student tools pages for exams, notifications and calculator.
- Teacher dashboard for class and schedule overview.
- Teacher classroom students page for roster, attendance and leave review.
- QR attendance page for scan-based attendance.
- Face enrollment page for preparing AI attendance.

5.11 Error Handling

The backend returns clear error messages when required fields are missing or operations fail. Email delivery uses retry settings and delivery logs. The frontend shows user-facing failure text when dashboard loading, leave submission, attendance processing or email tests fail. The AI service exposes health and readiness endpoints so infrastructure issues can be separated from application logic errors.

5.12 Configuration Management

Configuration values are stored in environment files and should be different for development and production. The report intentionally does not include actual database URLs, SMTP passwords or model secrets. A safe configuration example is included in the appendix using placeholders. This is important because academic documentation is often shared, printed or uploaded, and it must not expose real credentials.

This chapter covered the implementation approach. The next chapter explains the working of the main features from the user's point of view.

Chapter 6

Features and Working

This chapter explains the working of the system as a user would experience it. The purpose is to make the project understandable even for a reader who has not opened the code. Every major feature is described as a workflow: what the user does, what the backend does and what result appears.

6.1 Landing Page

The landing page introduces the project as a smart attendance platform rather than a simple register. It explains the main workflows: AI photo attendance, Today Attendance Data, class-wise student dashboard, exam eligibility, leave proof review, email alerts and teacher controls. It is useful for a first-time viewer because it gives a quick idea of the product before login.

The landing page also performs a health check so developers can see whether the backend is reachable. In a real deployment this helps distinguish a frontend loading issue from a backend service issue.

6.2 Registration and Login

Registration starts by selecting the role and entering profile details. The system sends an email OTP. The user enters the OTP and the account becomes verified. After verification, the user can log in and access the role-specific dashboard.

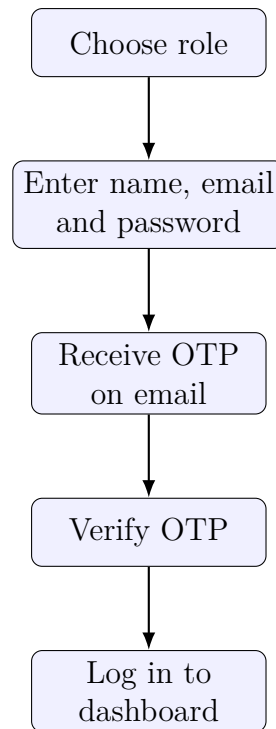


Figure 6.1: Registration and login working

6.3 Student Dashboard

The student dashboard is designed to answer useful student questions quickly. It shows the number of classes joined, total classes held, present count, absent count, attendance percentage, risk state, upcoming exams and notification count. Each class card opens a separate class page so the dashboard does not become crowded.

Table 6.1: Student dashboard feature matrix

Feature	What student sees	Why it is useful
Overview metrics	Joined classes, present, absent and percentage.	Converts attendance into simple academic status.
Class cards	Subject code, teacher, schedule and attendance health.	Lets student focus on one class at a time.
Notification bell	Count of exam, leave and attendance alerts.	Brings urgent items to attention.
Upcoming exams	Exam date, required percentage and eligibility.	Connects attendance to exam appearance risk.
Attendance calculator	Minimum future classes to attend.	Gives a clear plan instead of only a warning.
Face enrollment reminder	Profile readiness for AI attendance.	Reduces future recognition failures.
Leave request status	Pending, approved or rejected proof.	Avoids uncertainty after document upload.

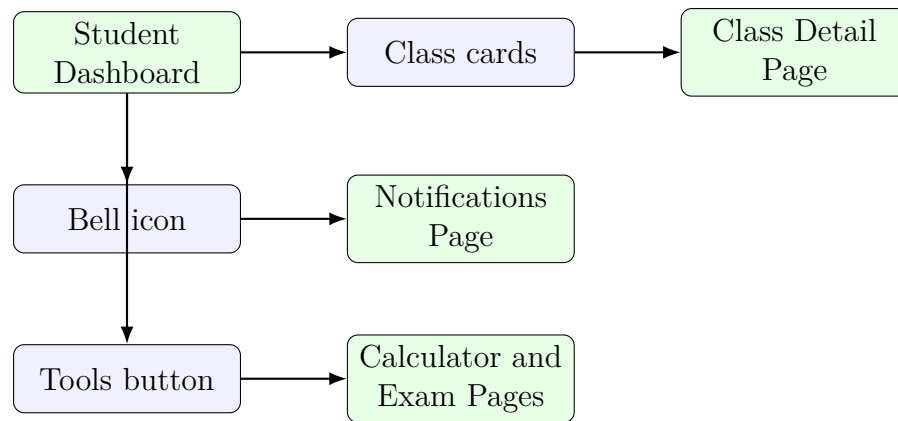


Figure 6.2: Student dashboard navigation flow

6.4 Student Class Detail Page

The class detail page is a focused page for one class. It shows attendance percentage, total sessions, missed sessions, recent records, regular schedule and class information. It also includes the leave proof form. The student can upload images or PDF files for medical reports, leave letters or absence reasons. The student can later see whether the request is pending, approved or rejected.

This separate page design is important. A student with many subjects should not have to read every detail on the main dashboard. The main dashboard gives summary and the class page gives depth.

6.5 Attendance Calculator

The attendance calculator is a planning tool. It takes current attendance percentage, current present and total counts, exam date, required eligibility percentage and expected number of future classes. When possible, future classes are estimated from the class schedule. The calculator then tells the student how many classes must be attended to reach the requirement safely.

Table 6.2: Attendance calculator inputs and outputs

Input or output		Meaning
Current count	present	Number of classes already attended.
Current count	total	Number of classes already recorded.
Current age	percent-	Existing attendance standing.
Exam date		Date until which future classes are estimated.
Required age	percent-	Eligibility threshold set by teacher or entered by student.
Future classes		Number of scheduled classes before the exam.
Minimum required attendance		Number of future classes the student should attend.
Safety message		Explains whether the target is possible, safe or risky.

6.6 Teacher Dashboard

The teacher dashboard is designed for repeated classroom work. It provides class creation, class list, schedule, exam setup and access to each class. A teacher can create a subject with code, section, semester and schedule. The generated join code allows students to join. The teacher can open a class to see students, attendance performance, leave requests and attendance controls.

6.7 Teacher Attendance Workflow

The teacher has multiple ways to take attendance. Manual attendance is useful for small classes or corrections. QR attendance is useful for quick self-marking during a time-limited session. AI photo attendance is useful when the teacher wants to capture classroom presence quickly from images.

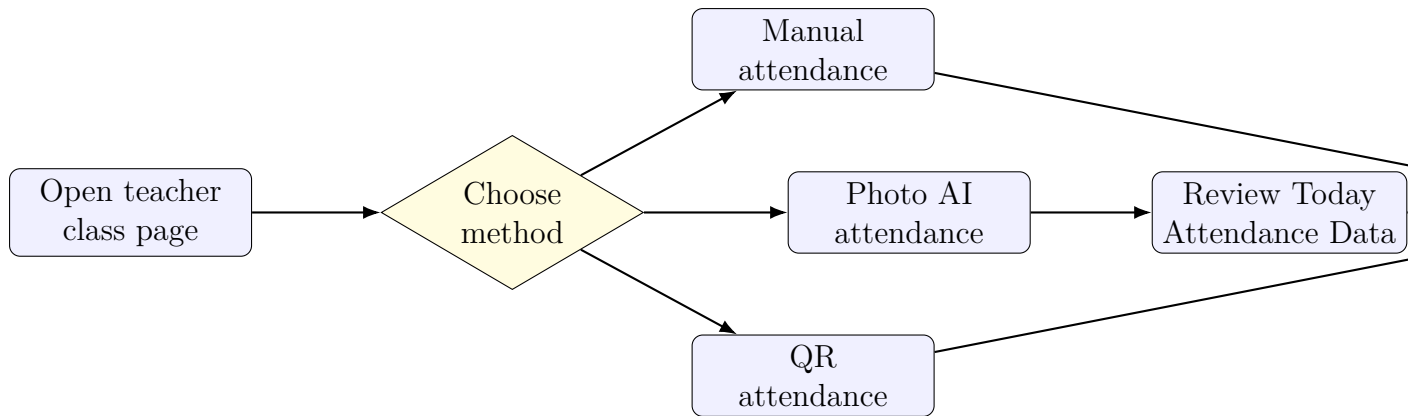


Figure 6.3: Teacher attendance workflow

6.8 AI Photo Attendance

In AI photo attendance, the teacher captures or uploads a classroom photo. The backend sends the photo to the AI service. The AI service detects faces and compares them with enrolled student face profiles. It returns matched students and verification notes. The teacher sees the result in a useful form: who is probably present, who is absent and which notes require attention.

The result goes to Today Attendance Data, not directly to final records. This is the key safety feature. Students who are marked absent receive an email early. If there is an error, the student can contact the teacher, and the teacher can correct the draft. This approach improves trust because the system is transparent and correctable.

6.9 Today Attendance Data Section

The Today Attendance Data section contains the current draft attendance for the day. It shows present students and absent students. The teacher can add a student to the present list, remove a student, change status and finalize. The draft can also be finalized automatically after the configured period, such as twelve hours, so that attendance does not remain pending forever.

Table 6.3: Today Attendance Data actions

Action	Who performs it	Result
Create draft	Teacher through photo attendance workflow	AI suggested attendance is stored for review.
Notify absentees	Backend email module	Students marked absent receive early email warning.
Add student	Teacher	Corrects false absent or missed recognition.
Remove student	Teacher	Corrects false present or accidental entry.
Finalize manually	Teacher	Draft becomes final attendance records.
Finalize automatically	Scheduler	Draft is submitted after the configured waiting period.

6.10 Leave Proof and Medical Report Workflow

The leave proof workflow solves a common issue: students often have a valid reason for absence but proof is shared informally through messages. In SAMS, proof is connected to the class and date. The student submits the request from the class detail page. The teacher previews the document in the student detail section and approves or rejects it. The student receives an email and dashboard status update.

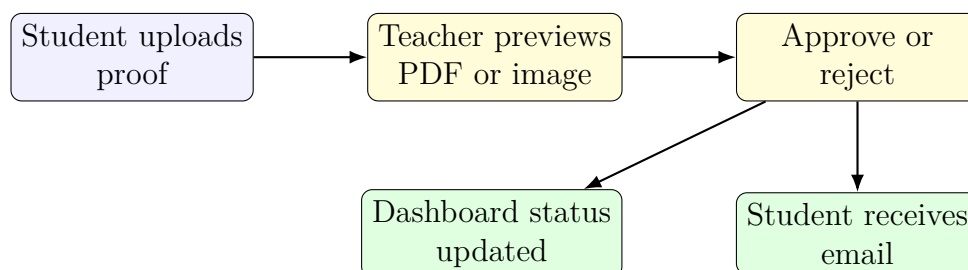


Figure 6.4: Leave proof review workflow

6.11 Exam Scheduling and Eligibility

The teacher can define an upcoming exam for a class. The exam entry contains exam date, required attendance percentage and optional notes. The student dashboard uses current attendance and schedule information to show eligibility. If the student is below the required percentage, the dashboard and email warnings explain the problem.

This feature is more useful than a normal attendance percentage because it connects present behaviour to future consequence. A student can see how many classes must be attended before

the exam. A teacher can warn students early instead of announcing ineligibility at the last moment.

6.12 Notification Bell and Separate Pages

The student dashboard includes a notification bell icon. It summarizes urgent items such as low attendance, upcoming exam risk, face enrollment reminder and leave request decision. The bell opens a separate notification page instead of expanding large content inside the dashboard. Similarly, tools such as the calculator and exams open on separate pages. This keeps the dashboard efficient and prevents it from becoming overloaded.

6.13 Class Cancellation and Reschedule Notice

If a class is cancelled or rescheduled, the teacher can update the class state. Students receive a bulk email notice with updated information. The attendance record can use a class-cancelled status so that cancelled sessions do not reduce student attendance percentage.

6.14 Assignments and Class Discussion

The project also includes class assignment and discussion features. Teachers can create assignments for a class, students can submit responses and class discussion messages can include replies or reactions. These features are useful because attendance systems are often part of a wider class management workflow. Keeping assignments and discussions in the same class context gives students one place to check academic activity.

6.15 Feature Summary

Table 6.4: Complete feature matrix

Feature	Student benefit	Teacher benefit
Email verification	Ensures account email can receive alerts.	Ensures teacher account can send and receive system notices.
Class join code	Easy class joining.	Less manual roster creation.
Class-wise dash-board	Understands subject-specific performance.	Reduces repeated student queries.

Feature	Student benefit	Teacher benefit
Photo attendance	Less roll-call time.	Faster attendance with review controls.
QR attendance	Fast self-marking when allowed.	Useful alternative to photo recognition.
Today Attendance Data	Early chance to report mistakes.	Correctable draft before final submission.
Leave proof upload	Formal way to submit medical or leave documents.	Online review without downloading every file.
Exam eligibility	Clear target before exam.	Early warning for at-risk students.
Attendance calculator	Personal planning tool.	Reduces confusion about required classes.
Notification bell	Urgent items in one place.	Students see warnings without teacher repeating them.
Email alerts	Important changes reach inbox.	Automated communication for repetitive events.
Cancellation notice	Knows class changes quickly.	Bulk class communication.

This chapter described the working features. The next chapter verifies these features through testing and result analysis.

Chapter 7

Testing and Results

Testing verifies that the system works as expected and that important academic workflows do not silently fail. For SAMS, testing is especially important because attendance data affects students directly. The project therefore uses a combination of unit tests, service checks, manual workflow tests and document-level validation.

7.1 Testing Strategy

The testing strategy is based on risk. Features that can affect attendance records, email delivery or eligibility warnings require more careful checking. UI-only improvements can be checked through browser interaction and visual review. AI recognition pipelines need separate tests because they depend on model readiness, image input and stored face profiles.

Table 7.1: Testing levels used for the project

Level	Purpose	Examples
Unit testing	Verify a small service or function.	AI face recognition service tests and attendance calculation logic.
Integration testing	Verify that multiple modules work together.	Backend creates attendance draft after AI result; email service records delivery state.
API testing	Verify request and response behaviour.	Dashboard routes, signup verification, leave request submission and draft finalization.
UI testing	Verify pages and workflows in the browser.	Student dashboard, notification page, teacher class page and leave proof preview.
Operational testing	Verify service readiness.	Backend health endpoint, AI readiness endpoint and email status endpoint.
Security review	Verify no sensitive data is exposed.	Report and status endpoints do not reveal database URI or SMTP password.

7.2 Backend API Test Scenarios

Backend API tests focus on data correctness and error handling. Every endpoint should reject incomplete input, return useful messages and not expose internal secrets. Teacher-only operations should be scoped to teacher and class ids.

Table 7.2: Representative backend API test scenarios

Scenario	Test action	Expected result
Health check	Call backend health endpoint.	Response shows service status and safe dependency state.
Email status	Call email status endpoint.	Response shows enabled and verification information without password.
Signup	Register student or teacher.	User is created in verification-required state and OTP email is queued.
Email verification	Submit valid OTP.	Email is marked verified and welcome email is sent.
Login before verification	Attempt login with unverified account.	Backend prevents full access or asks for verification.

Scenario	Test action	Expected result
Forgot password	Request reset.	Reset OTP or token email is sent with expiry.
Class creation	Teacher submits valid class data.	Class is created and join code is generated.
Student join class	Student submits valid join code.	Membership is created and appears in dashboard.
Manual attendance	Teacher submits attendance list.	Attendance records are written with correct status.
Photo attendance	Teacher submits classroom image.	AI result becomes Today Attendance Data draft.
Draft update	Teacher adds or removes student in draft.	Draft list changes but final records are not written yet.
Draft finalization	Teacher finalizes draft.	Final attendance records are written and draft becomes finalized.
Leave request	Student uploads reason and proof.	Leave request appears in teacher student detail section.
Leave review	Teacher approves or rejects.	Request status changes and student email is queued.
Exam setup	Teacher sets exam date and required percentage.	Student dashboard shows upcoming exam and eligibility.
Class cancellation	Teacher cancels today's class.	Students receive email and attendance is not counted as absence.

7.3 Frontend Workflow Test Scenarios

Frontend tests ensure that users can complete workflows without confusion. The student dashboard was reviewed to ensure that key tools open on separate pages. The notification bell should be visible and should lead to a notification view. Class cards should open class details. The teacher workflow should allow attendance review before finalization.

Table 7.3: Frontend workflow testing

Page	Check	Expected result
Landing page	Feature explanation and health state.	New visitor understands capabilities and backend status.
Signup page	OTP verification flow.	User sees verification step and helpful errors.
Student dashboard	Overview metrics and navigation.	Counts, class cards, bell icon and tool links are visible.
Student tools page	Calculator and exam views.	Tool opens as a separate page and computes useful guidance.
Student class detail	Leave proof upload and history.	Student can submit proof and inspect previous attendance.
Teacher dashboard	Class overview and schedule.	Teacher can create classes and open class management.
Teacher class students	Roster, analytics and leave review.	Teacher can filter students and preview proof files online.
Attendance workspace	Draft creation and finalization.	Teacher can review Today Attendance Data before final submission.

7.4 AI Service Testing

The AI service contains tests for face recognition service behaviour and classroom attendance pipeline behaviour. These tests check that the service can process expected input shapes, return structured results and handle degraded or missing data cases. AI testing is not only about matching faces; it also verifies that the application can continue gracefully when models are unavailable or input quality is poor.

Table 7.4: AI service test concerns

Concern	Expected behaviour
Model readiness	Health and readiness endpoints distinguish process health from model availability.
Face detection	Service identifies faces from classroom image input when model assets are available.
Face matching	Embeddings are compared with enrolled profiles and confidence notes are returned.
Unmatched faces	Unknown or low-confidence faces are reported without corrupting attendance records.
Missing enrollment	Student without face profile is not falsely assumed present by the AI pipeline.
Gateway timeout	Backend can report AI failure instead of hanging indefinitely.

7.5 Email Testing

Email testing has two parts. The first part checks configuration: SMTP host, port, secure mode, sender identity and credentials. The second part checks notification events. The backend contains safe endpoints for email status, SMTP verification and protected test email sending. Delivery logs help identify whether a message was sent, retried or failed.

Table 7.5: Email notification testing scenarios

Email type	Trigger tested	Expected result
Verification OTP	New signup or resend verification.	OTP email is delivered and expires after configured minutes.
Welcome email	Successful verification.	User receives professional account activation message.
Password reset	Reset request from login screen.	Reset OTP or link is sent and cannot be used after expiry.
Profile email OTP	Email update in profile.	New email receives OTP before profile email changes.
Leave approval	Teacher approves request.	Student receives approval email with leave date and class.
Leave rejection	Teacher rejects request.	Student receives rejection email and optional teacher note.
Exam warning	Student below required attendance.	Student receives current and required percentage warning.
Absent notice	Attendance draft marks student absent.	Student and optional parent receive subject, date and timing context.
Cancellation	Teacher cancels or reschedules class.	Roster receives updated class information.

7.6 Attendance Draft Testing

The Today Attendance Data feature requires special tests because it sits between AI output and final attendance records. The most important result is that an AI session should create a draft, not immediately finalize attendance. The teacher must be able to correct the draft. Final records should only appear after manual finalization or automatic scheduler finalization.

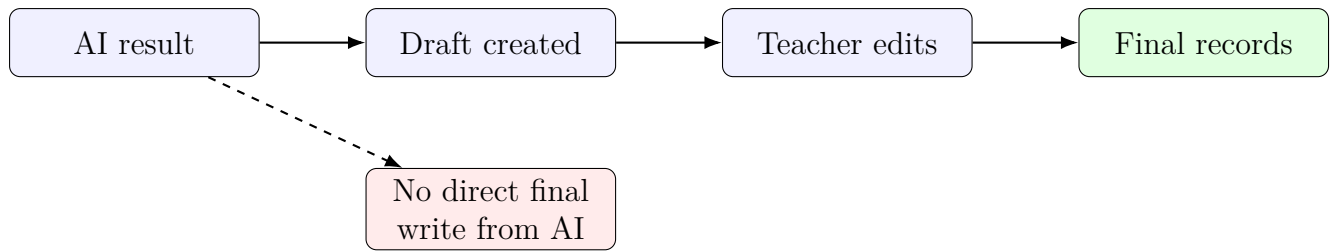


Figure 7.1: Attendance draft testing focus

The draft tests check the following outcomes:

- Draft contains class id, teacher id, present list, absent list and source information.
- Absentee emails are triggered when the draft is created.
- Teacher can add a student who was missed by AI.
- Teacher can remove a student incorrectly marked present.
- Finalization creates one final attendance entry per roster student for the session.
- Finalized draft cannot be accidentally finalized again.

7.7 Security and Privacy Checks

Security testing for this project focused on preventing accidental exposure. The report, frontend and safe status endpoints must not show MongoDB connection strings, Gmail app passwords or other credentials. OTP values should not appear in production responses. Email delivery logs should store useful metadata and errors without storing secrets.

Document proof also needs careful handling. The teacher should preview files from the application, but the system should avoid encouraging unnecessary downloading. In a production version, file upload should include size limits, type validation and scanning. For this academic implementation, the report documents the expected controls and the application keeps the review workflow inside the class detail context.

7.8 Result Analysis

The implemented project satisfies the core goal of making attendance more transparent and useful. Students can see class-wise attendance and eligibility information rather than only a final percentage. Teachers can run different attendance methods and correct AI results before final submission. Email communication reduces manual reminders. Leave proof connects absence reason to class records.

The main improvement over a traditional register is not only speed; it is feedback. Students receive early absence notifications, exam warnings and leave decision emails. Teachers receive a manageable review workflow. The system also avoids the dangerous assumption that AI output should always be final.

7.9 Known Limitations Found During Testing

Some limitations remain. AI accuracy depends on enrollment quality, lighting, camera angle and image resolution. Email delivery depends on SMTP provider settings and account limits. A real institute deployment would need stronger authentication middleware, role-based authorization on every route, production file storage, stricter upload validation, rate limiting and backups. These limitations do not invalidate the project; they show the difference between a final-year prototype and an institute-wide production rollout.

7.10 Testing Conclusion

Testing shows that the project has a coherent full-stack workflow: registration verifies email, classes can be created and joined, attendance can be captured and reviewed, students can track analytics, teachers can review proof, exams can generate eligibility guidance and emails can notify users. The project is therefore suitable as a final academic project and as a foundation for future production hardening.

Chapter 8

Deployment, Limitations and Future Scope

This chapter explains how the project can be run, deployed and improved. SAMS is implemented as a multi-service application, so deployment means making the frontend, backend, AI service, database and email provider work together reliably.

8.1 Local Development Deployment

In local development, the project can be started from the repository root. The frontend runs as a React development server, the backend runs as an Express server and the AI service runs with Uvicorn. MongoDB can be provided locally or through Docker.

Table 8.1: Local service deployment summary

Service	Typical command	Purpose
Frontend	<code>npm run dev:frontend</code>	Starts the React interface for browser use.
Backend	<code>npm run dev:backend</code>	Starts the REST API and connects to MongoDB.
AI service	<code>npm run dev:ai</code> or Uvicorn	Starts FastAPI face recognition endpoints.
MongoDB	Docker Compose or local MongoDB	Stores users, classes, attendance and logs.
Email SMTP	Configured through backend environment	Sends OTPs and academic notifications.

For a development machine, it is common to run the backend and frontend first, then start the AI service when testing face enrollment or photo attendance. This reduces setup time when working only on dashboards or email templates.

8.2 Environment Configuration

The backend uses environment variables for database, email and AI settings. Sensitive values must never be committed or printed in reports. A safe example is provided in Appendix A. In production, values such as database URI and SMTP password should be set through the hosting provider’s secret manager.

Table 8.2: Important environment categories

Category	Examples of values
Server	Port, frontend origin and API base URL.
Database	MongoDB connection string and database name.
AI gateway	AI service base URL and timeout values.
Email	SMTP host, port, sender, username, password and retry settings.
OTP	Expiry time and development exposure flag.
Model runtime	Face model pack, provider and asset directory.

8.3 Production Deployment View

A production deployment can place the frontend on a static hosting platform, the backend on a Node.js hosting platform, the AI service on a Python-capable host and MongoDB on a managed database service. The backend should be the only service exposed to the frontend. The AI service should be accessible to the backend and protected from direct public misuse when possible.

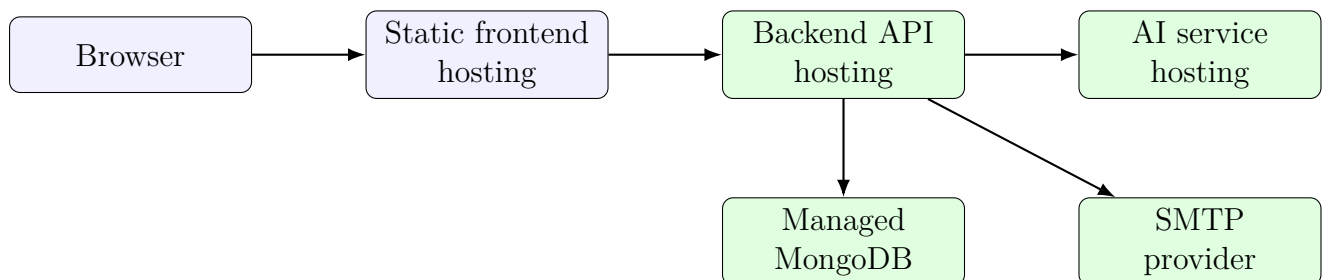


Figure 8.1: Production deployment view

8.4 Operational Monitoring

The project includes health and status concepts that are important for operations. The backend health endpoint can show whether the backend is running and whether dependencies are degraded. The AI service has a readiness endpoint so model loading can be checked separately from process uptime. The email status endpoint can verify SMTP connectivity without revealing credentials.

In a future production system, operational monitoring should include request logs, attendance finalization logs, email bounce tracking, uptime monitoring, database backups and alerts for repeated AI or SMTP failures.

8.5 Limitations

Every project has limitations. Understanding them is important because it prevents unrealistic expectations.

Table 8.3: Current limitations

Area	Limitation
AI recognition	Face matching can fail due to poor lighting, blur, occlusion, side angle or incomplete enrollment.
Manual review	Teacher review improves reliability but still depends on teacher attention before final submission.
Email delivery	Free SMTP or Gmail app passwords may have sending limits and provider-specific security rules.
File uploads	A production deployment should add stronger file type validation, file size limits and malware scanning.
Authorization	A production institute system should add strict middleware checks for every role and route.
Scalability	Large institutes may need queues for AI jobs, background workers and caching.
Privacy	Face embeddings and proof documents require formal data retention and consent policies.
Network dependency	Real-time email and AI requests depend on network connectivity.

8.6 Future Scope

The project can be extended in several directions. Some improvements are technical, while others are academic workflow enhancements.

Table 8.4: Future enhancement roadmap

Priority	Enhancement	Expected value
High	Strong RBAC middleware	Ensures every route verifies role, identity and class ownership.
High	Production file storage	Stores leave documents in object storage with signed access.
High	Attendance audit trail	Keeps a history of every draft edit and finalization action.
Medium	Background AI queue	Handles large photo attendance jobs without blocking requests.
Medium	Parent portal	Gives parents controlled visibility into attendance and alerts.
Medium	Reports export	Generates PDF or spreadsheet reports for department records.
Medium	Calendar integration	Syncs class schedule and exam dates with external calendars.
Low	Mobile app	Improves camera capture, QR scanning and push notifications.
Low	Advanced analytics	Predicts future risk patterns and recommends interventions.

8.7 Ethical and Privacy Considerations

Because the project uses face recognition and academic data, ethics must be considered. Students should know why face enrollment is collected and how it is used. The system should store only the data required for attendance. Access to face profiles, attendance data and medical documents should be limited. A production institution should define retention periods and deletion policy.

The human review design is also an ethical choice. AI output is not treated as unquestionable truth. The teacher reviews Today Attendance Data, students receive early absence notices and corrections are possible before final submission. This makes the system more fair than a fully automatic black-box attendance system.

8.8 Conclusion

The Smart Attendance Management System provides a complete modern attendance workflow. It combines role-based dashboards, class management, AI-assisted attendance, QR and manual attendance, Today Attendance Data, student analytics, leave proof review, exam eligibility planning and email notifications. The project is implemented with a practical full-stack architecture using React, Express, MongoDB and FastAPI.

The most valuable contribution of the project is the combination of automation and review. AI reduces repetitive classroom work, but final attendance remains correctable. Students get useful information instead of only a final number. Teachers get tools to manage attendance, leave and exam eligibility in one place. With stronger production security, file storage and background processing, the system can become a robust institute-level attendance platform.

Appendix A

Supplementary Material

This appendix gives practical reference material for readers who want to understand the project setup, API surface and data model quickly. All environment examples are sanitized. Real database URLs, SMTP passwords and app passwords must never be written in documentation.

A.1 Sanitized Environment Setup

A.1.1 Backend Environment Example

```
NODE_ENV=development
PORT=4000
FRONTEND_ORIGIN=http://localhost:5173
MONGODB_URI=mongodb://localhost:27017/sams

AI_SERVICE_BASE_URL=http://localhost:8000
AI_REQUEST_TIMEOUT_MS=120000
AI_HEALTH_TIMEOUT_MS=10000

EMAIL_ENABLED=true
EMAIL_FROM="SAMS Notifications <your-address@example.com>"
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=465
EMAIL_SECURE=true
EMAIL_USER=your-address@example.com
EMAIL_PASSWORD=your-app-password
EMAIL_RETRY_COUNT=2
EMAIL_RETRY_DELAY_MS=2000
EMAIL_QUEUE_CONCURRENCY=3
```

```
EMAIL_TEST_RECIPIENT=your-test-recipient@example.com
EMAIL_TEST_KEY=change-this-test-key
EMAIL_OTP_EXPIRES_MINUTES=10
EMAIL_EXPOSE_OTP_IN_RESPONSE=false
```

A.1.2 AI Service Environment Example

```
MODEL_DEVICE=cpu
FACE_ANALYSIS_PROVIDERS=CPUExecutionProvider
FACE_ANALYSIS_MODEL_PACK=antelopev2
FACE_ANALYSIS_MODEL_PACKS=antelopev2,bufferlo_1
FACE_ANALYSIS_AUTO_DOWNLOAD=true
```

A.2 Common Development Commands

Table A.1: Common development commands

Command	Purpose
<code>npm run dev</code>	Start multiple project services from the monorepo when configured.
<code>npm run dev:frontend</code>	Start the React frontend development server.
<code>npm run dev:backend</code>	Start the Express backend server.
<code>npm run dev:ai</code>	Start the FastAPI AI service through the project script.
<code>npm -workspace backend install</code>	Install backend Node.js dependencies.
<code>npm -workspace frontend install</code>	Install frontend Node.js dependencies.
<code>pip install -r ai-service/requirements.txt</code>	Install AI service Python dependencies.
<code>docker compose up</code>	Start services defined in Docker Compose, including MongoDB when configured.

A.3 API Endpoint Summary

The following table summarizes the major REST endpoints visible from the current backend router. It is not a substitute for generated API documentation, but it helps a reader understand the project scope.

Table A.2: API endpoint summary

Method	Endpoint	Purpose
GET	/api/v1/health	Backend health and dependency status.
GET	/api/v1/email/status	Safe email service status.
POST	/api/v1/email/verify	Verify SMTP connection.
POST	/api/v1/email/test	Send protected test email.
GET	/api/v1/attendance/summary	General attendance summary.
POST	/api/v1/attendance/verify	Attendance verification endpoint.
POST	/api/v1/auth/signup	Register user and start verification.
POST	/api/v1/auth/login	Log in user.
POST	/api/v1/auth/verify-email	Verify email OTP.
POST	/api/v1/auth/resend-verification	Resend signup verification OTP.
POST	/api/v1/auth/password-reset/request	Request password reset OTP.
POST	/api/v1/auth/password-reset/confirm	Confirm password reset.
POST	/api/v1/users/:role/:userId/email-otp	Send OTP for profile email update.
PATCH	/api/v1/users/:role/:userId/profile	Update user profile after validation.
GET	/api/v1/classes/:classId/assignments	List class assignments.
GET	/api/v1/classes/:classId/discussions	List class discussion messages.
POST	/api/v1/classes/:classId/discussions	Post a class discussion message.
POST	/api/v1/classes/:classId/discussions/:messageId/replies	Add new reply to a discussion message.
POST	/api/v1/classes/:classId/discussions/:messageId/reactions	React to a discussion message.
GET	/api/v1/students/:userId/dashboard	Load student dashboard summary.
POST	/api/v1/students/:studentId/class/:classId/join	Student join a class.
POST	/api/v1/students/:studentId/class/:classId/assignments/:assignmentId/submit	Student submit assignment.
POST	/api/v1/students/:studentId/class/:classId/leave-requests	Student class leave requests.
GET	/api/v1/students/:studentId/faculty/enroll	Load faculty enrollment profile state.
POST	/api/v1/students/:studentId/faculty/enroll	Enroll student to faculty profile.
GET	/api/v1/teachers/:userId/dashboard	Load teacher dashboard summary.

Method	Endpoint	Purpose
POST	/api/v1/teachers/:teacherId/classes	Teacher creates a class.
POST	/api/v1/teachers/:teacherId/classes/:classId/assignments	Teacher creates assignments.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/examrequired	Set exam required percentage.
GET	/api/v1/teachers/:teacherId/classes/:classId	Class detail.
POST	/api/v1/teachers/:teacherId/classes/:classId/students	Adds student to class.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/students/:studentId	Update class student.
DELETE	/api/v1/teachers/:teacherId/classes/:classId/students/:studentId	Remove student from class.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/manual	Set class attendance manually.
POST	/api/v1/teachers/:teacherId/classes/:classId/cancel-today	Cancel today's class.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/qrcode	Generate QR session.
POST	/api/v1/students/attendance/qrcode	Student marks attendance through QR scan.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session	Teacher creates attendance session draft.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId	Update attendance session draft.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId/attendance	Teacher adds attendance to session draft.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId/attendance/qr	Teacher adds QR attendance to session draft.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId/attendance/qr/attendance	Teacher adds QR attendance to session draft.
POST	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId/attendance/qr/attendance/attendance	Teacher adds QR attendance to session draft.
PATCH	/api/v1/teachers/:teacherId/classes/:classId/attendance/session/:sessionId/attendance/qr/attendance/attendance/attendance	Teacher adds QR attendance to session draft.

A.4 Model and Collection Reference

Table A.3: Model reference

Model	Main fields and meaning
User	user id, role, name, email, parent email, email verified flag and profile fields.
EmailOtp	email, purpose, OTP hash or value metadata, expiry and used state.
Classroom	class id, teacher id, subject name, subject code, section, room, semester and schedule slots.
ClassMembership	class id, student id, membership state and timestamps.

Model	Main fields and meaning
AttendanceRecord	class id, student id, student name, status, source, recorded date and notes.
AttendanceDraft	class id, teacher id, draft records, source, created time, finalize time and final status.
QrAttendanceSession	class id, session code, expiry, active state and scan information.
FaceProfile	student id, enrollment state and face recognition metadata.
LeaveRequest	class id, student id, absence date, reason, type, attachments, status and teacher note.
ClassExam	class id, exam title, exam date, required attendance percentage and notes.
EmailDeliveryLog	recipient, template name, notification key, status, attempts and error summary.
ClassAssignment	class id, teacher id, title, description, due date and attachment metadata.
ClassAssignmentSubmission	assignment id, class id, student id, submitted content and status.
ClassDiscussionMessage	class id, author, message, replies, reactions and timestamps.

A.5 Sample Email Template Content

The exact design of email templates can be changed, but every template should include clear subject, greeting, dynamic class or account information and a short action line. The following examples are intentionally generic.

Table A.4: Sample email subjects

Event	Sample subject line
Signup verification	Verify your SAMS account
Welcome message	Welcome to MarkIn / SAMS
Password reset	Reset your SAMS password
Profile email verification	Confirm your new profile email
Leave approved	Your leave request has been approved
Leave rejected	Your leave request has been reviewed
Exam attendance warning	Attendance warning for upcoming exam
Absent notification	You were marked absent today
Class cancellation	Class cancelled or rescheduled

A.6 Report Figure Inventory

Table A.5: Important diagrams included in the report

Diagram	Purpose
Overall architecture	Shows frontend, backend, AI service, database and email relationship.
Service communication	Explains API calls and data flow between services.
Entity relationship	Explains MongoDB collections and relationships.
Today Attendance Data flow	Shows draft creation, absentee alert, correction and finalization.
Email notification flow	Shows event, template, queue, SMTP and delivery log.
Student dashboard flow	Shows dashboard to class, notification and tool pages.
Teacher attendance flow	Shows manual, QR and AI photo attendance routes.
Leave proof flow	Shows upload, preview, approve or reject and notification.
Deployment view	Shows production hosting arrangement.

A.7 Glossary

Table A.6: Glossary

Term	Meaning
SAMS	Smart Attendance Management System.
MarkIn	Product name used for the SAMS application.
Attendance draft	Temporary attendance data created before final submission.
Today Attendance Data	Teacher review section for current draft attendance.
Face enrollment	Process where a student registers face data for AI attendance.
OTP	One-time password sent by email for verification or password recovery.
Eligibility percentage	Minimum attendance required to appear in an exam.
Roster	List of students enrolled in a class.
SMTP	Standard protocol used for sending email through a mail provider.
Mongoose	MongoDB object modelling library used in the backend.
FastAPI	Python framework used for the AI service.

Bibliography

- [1] MongoDB, Express, React and Node.js documentation, “MERN stack application development,” online documentation, accessed May 2026.
- [2] FastAPI documentation, “FastAPI framework, high performance APIs with Python,” online documentation, accessed May 2026.
- [3] InsightFace project documentation, “2D and 3D face analysis toolkit,” online documentation, accessed May 2026.
- [4] J. Deng, J. Guo, N. Xue and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [5] D. Guo et al., “Sample and Computation Redistribution for Efficient Face Detection,” SCRFD face detection model documentation and associated research resources, accessed May 2026.
- [6] Nodemailer documentation, “Send emails from Node.js,” online documentation, accessed May 2026.
- [7] MongoDB documentation, “Document data model, indexing and query operations,” online documentation, accessed May 2026.
- [8] React documentation, “Building user interfaces with components,” online documentation, accessed May 2026.
- [9] Project source repository documentation, *Smart Attendance Management System README, architecture notes and email notification notes*, local project documentation, 2026.